



E.E.S.T 1 | 5°3° 2026

Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Actividad 5: Modelado de Datos con Struct y Punteros

C++

Repositorio de GitHub

<https://github.com/Martuuuuuuuu/DATOSESTRUCTURADOS.git>

Participantes

Martina Araujo, Thiago Goya, Sofia Salaberry

Escuela

Escuela Técnica N°1 de Vicente López

Curso

Ciclo Superior 5°3° turno tarde

Docentes a cargo

Yamil Ganduglia, York Elias Mansilla Muñoz

Año:

2026



E.E.S.T 1 | 5°3° 2026

Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Índice

Introducción.....	3
Objetivos.....	3
Estructura de Carpetas.....	3
Desarrollo de la actividad.....	4
Capturas del código fuente.....	5
Capturas de ejecución.....	6
División de tareas.....	6
Cuestionario.....	7
Respuestas.....	7
Conclusión.....	8



Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Introducción

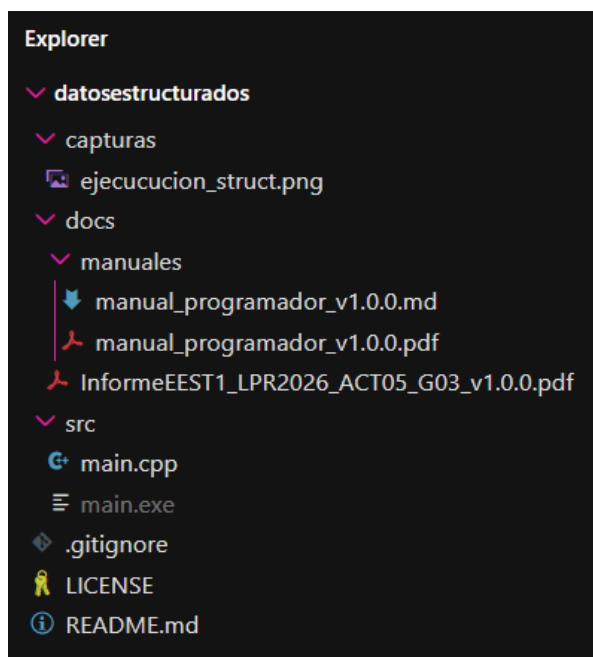
En esta actividad trabajamos con el modelado de datos mediante estructuras (struct) y punteros en C++. El objetivo principal fue aprender a agrupar diferentes tipos de datos dentro de una misma estructura, representando una entidad de manera más organizada. También aprendimos a utilizar punteros para acceder y modificar los datos de una estructura mediante el operador flecha (->).

Además, trabajamos con el pasaje de datos por referencia y con el manejo del buffer de entrada utilizando `cin.ignore()`. Estos conceptos nos permitieron entender un poco mejor cómo se almacenan y modifican los datos en la memoria mientras se ejecuta un programa.

Objetivos

El objetivo de esta actividad fue aprender a crear y utilizar estructuras (struct) en C++ para agrupar diferentes datos relacionados dentro de una misma entidad. También buscamos comprender cómo funcionan los punteros, cómo pasar la dirección de una variable a una función y cómo utilizar el operador flecha (->) para acceder y modificar los datos de una estructura. Además, trabajamos con `cin.ignore()` para limpiar el buffer de entrada antes de utilizar `cin.getline()`.

Estructura de Carpetas





Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Desarrollo de la actividad

Durante la actividad creamos una estructura llamada EntidadProyecto para poder agrupar distintos datos relacionados dentro de una misma entidad. La estructura utilizada contiene un identificador, un nombre y una métrica.

```
// 1. DEFINICIÓN DE LA ESTRUCTURA (Modelo sugerido para el Proyecto Integrador Anual)
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica; // Representa lecturas de sensores, consumo, avance, etc.
};
```

Después, en la función main(), se creó una variable de tipo EntidadProyecto. A partir de esta variable se pasó su dirección a una función mediante el operador &.

```
cargarDatos(&miEntidad);
```

La función recibe esa dirección mediante un puntero, lo que permite trabajar directamente sobre la estructura original.

```
void cargarDatos(EntidadProyecto* ptr) {
```

Para acceder a los miembros de la estructura desde el puntero utilizamos el operador flecha (->). Por ejemplo:

```
cout << "=> Ingrese el ID de la entidad (entero): ";
cin >> ptr->id;

// Limpieza del buffer obligatoria antes de cin.getline
cin.ignore();

cout << "=> Ingrese el Nombre o Descripcion: ";
cin.getline(ptr->nombre, 50);

cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";
cin >> ptr->metrica;
}
```

También utilizamos cin.ignore() antes de cin.getline() para evitar problemas con el salto de línea que queda en el buffer después de ingresar datos mediante cin. De esta manera, pudimos ingresar correctamente un nombre completo, incluso cuando contiene espacios.



Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Capturas del código fuente

```
src > main.cpp > main()
1  /*
2  =====
3  Materia: Laboratorio de Programación (LPR) - 5°
4  Archivo: src/main.cpp
5  Actividad 5: Modelado de datos con Structs y Punteros
6  =====
7  */
8
9  #include <iostream>
10 #include <cstring>
11
12 using namespace std;
13
14 // 1. DEFINICIÓN DE LA ESTRUCTURA (Modelo sugerido para el Proyecto Integrador Anual)
15 struct EntidadProyecto {
16     int id;
17     char nombre[50];
18     float metrica; // Representa lecturas de sensores, consumo, avance, etc.
19 };
20
21 // Prototipo de función con pasaje por dirección mediante puntero
22 void cargarDatos(EntidadProyecto* ptr);
23
24 int main() {
25     // Inicialización de seguridad en la memoria Stack
26     EntidadProyecto miEntidad = {0, "Vacío - [Martina Araujo]", 0.0f};
27
28     cout << "===== " << endl;
29     cout << " MODELADO STRUCT - ESTUDIANTE: [Martina Araujo] " << endl;
30     cout << "===== " << endl;
31
32     // Invocación enviando la dirección física de memoria con '&'
33     cargarDatos(&miEntidad);
34
35     cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
36     cout << "ID Registrado: " << miEntidad.id << endl;
37     cout << "Nombre Registrado: " << miEntidad.nombre << endl;
38     cout << "Métrica Guardada: " << miEntidad.metrica << endl;
39     cout << "Dirección RAM Hexadecimal: " << &miEntidad << endl;
40     cout << "===== " << endl;
41
42     return 0; // Código 0: Finalización exitosa
43 }
44
45 void cargarDatos(EntidadProyecto* ptr) {
46     cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;
47
48     cout << "=> Ingrese el ID de la entidad (entero): ";
49     cin >> ptr->id;
50
51     // Limpieza del buffer obligatoria antes de cin.getline
52     cin.ignore();
53
54     cout << "=> Ingrese el Nombre o Descripción: ";
55     cin.getline(ptr->nombre, 50);
56
57     cout << "=> Ingrese la Métrica de Operación (decimal/float): ";
58     cin >> ptr->metrica;
59 }
60
```



Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Capturas de ejecución

```
PS C:\Users\Martina\Desktop\datosestructurados> cd src
PS C:\Users\Martina\Desktop\datosestructurados\src> .\main.exe

=====
MODELADO STRUCT - ESTUDIANTE: [Martina Araujo]
=====

--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---
=> Ingrese el ID de la entidad (entero): 2
=> Ingrese el Nombre o Descripcion: Siri
=> Ingrese la Metrica de Operacion (decimal/float): 45

=== DATOS VERIFICADOS EN LA MEMORIA RAM ===
ID Registrado:      2
Nombre Registrado:  Siri
Metrica Guardada:   45
Direccion RAM Hexadecimal: 0xbbb9dffad0
=====
PS C:\Users\Martina\Desktop\datosestructurados\src> |
```

División de tareas

Participantes	Tarea realizada
Martina Araujo	Informe, Readme, Capturas y código
Thiago Goya	Readme, License e Informe, código
Sofia Salaberry	Manual e Informe, código

Cuestionario

CUESTIONARIO TÉCNICO DE CONTROL

Respondan las siguientes 3 preguntas dentro del informe PDF final que entregarán en Moodle.



Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry



Pregunta 1: Diferencia entre & y *

- **Explicación:** El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- **Pregunta:** ¿Qué valor imprime en pantalla la instrucción `cout << p;` comparada con `cout << *p;`?



Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Intentar escribir `cin >> p;` sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- **Pregunta:** ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar `cin` para modificar el valor de la variable original?



Pregunta 3: Formato Hexadecimal

- **Explicación:** Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) anteceditas por 0x (ej: 0x61fe1c).
- **Pregunta:** ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Respuestas

1. Al usar “p”, se muestra la dirección de memoria donde está guardada la variable edad, en cambio, cuando usamos “*p”, mostramos el valor que está guardado en esa dirección, por ejemplo, “p” puede mostrar un valor como 0x61fe1c, mientras que “*p” mostraría la edad que ingresamos.

2. Es obligatorio usar “*p” porque “p” guarda la dirección de memoria de la variable edad, al poner “`cin >> *p`”, el número que ingresamos se guarda en edad a través del puntero, si ponemos “`cin >> p`”, estaríamos intentando guardar el dato en el puntero y no en la variable original.

3. Las direcciones de memoria se muestran en hexadecimal porque es una forma más corta y sencilla de representar números grandes, además, el hexadecimal se relaciona fácilmente con la forma en que la computadora trabaja con los bits y la memoria.



Actividad 5: Datos estructurados

Martina Araujo, Thiago Goya, Sofia Salaberry

Conclusión

En conclusión, esta actividad nos permitió comprender mejor cómo se pueden organizar los datos en C++ mediante el uso de estructuras (struct) y cómo se pueden modificar esos datos desde una función utilizando punteros. También aprendimos a diferenciar el uso del operador . y del operador flecha (->), entendiendo que este último se utiliza cuando trabajamos con un puntero a una estructura. Además, aprendimos la función de `cin.ignore()` y su importancia al momento de utilizar `cin.getline()`.